7CS034 Data Science

Week 2 Data Transformation

Dr. Liam Naughton

1 Introduction

Last time we learned about the fundamentals of data science and we spent some time in the workshop building our own visualizations using `ggplot2`. Visualization is an important tool for generating insight, but it's rare that you get the data in exactly the right form you need to make the graph you want. Often you'll need to create some new variables or summaries to answer your questions with your data, or maybe you just want to rename the variables or reorder the observations to make the data a little easier to work with. You'll learn how to do all that (and more!) in this section, which will introduce you to data transformation using the `dplyr` package and a dataset on flights that departed from New York City in 2013.

The goal of this section is to give you an overview of all the key tools for transforming a data frame. We'll start with functions that operate on rows and then columns of a data frame, then circle back to talk more about the pipe, an important tool that you use to combine verbs. We will then introduce the ability to work with groups. We will end the section with a case study that showcases these functions in action and we'll come back to the functions in more detail in later sections, as we start to dig into specific types of data (e.g., numbers, strings, dates).

We'll focus on the `dplyr` package, another core member of the tidyverse. We'll illustrate the key ideas using data from the `nycflights13` package, and use `ggplot2` to help us understand the data.

```
library(nycflights13)
library(tidyverse)
```

```
Type 'q()' to quit R.

> library(nycflights13)
> library(tidyverse)
── Attaching packages ─────────────── tidyverse 1.3.2 ──
✔ ggplot2 3.3.6     ✔ purrr   0.3.5
✔ tibble  3.1.7     ✔ dplyr   1.0.10
✔ tidyr   1.2.1     ✔ stringr 1.4.1
✔ readr   2.1.3     ✔ forcats 0.5.2
── Conflicts ────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
>
```

Take careful note of the conflicts message that's printed when you load the `tidyverse`. It tells you that `dplyr` overwrites some functions in base `R`. If you want to use the base version of these functions after loading `dplyr`, you'll need to use their full names: `stats::filter()` and `stats::lag()`. So far we've mostly ignored which package a function comes from because most of the time it doesn't matter. However, knowing the package can help you find help and find related functions, so when we need to be precise about which package a function comes from, we'll use the same syntax as R: `packagename::functionname()`.

2 NYCFLIGHTS13

To explore the basic `dplyr`verbs, we're going to use `nycflights13::flights`. This dataset contains all 336,776 flights that departed from New York City in 2013. The data comes from the US Bureau of Transportation Statistics, and is documented in `?flights`.

```
flights
```

```
> flights
# A tibble: 336,776 × 19
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      517            515         2
 2  2013     1     1      533            529         4
 3  2013     1     1      542            540         2
 4  2013     1     1      544            545        -1
 5  2013     1     1      554            600        -6
 6  2013     1     1      554            558        -4
 7  2013     1     1      555            600        -5
 8  2013     1     1      557            600        -3
 9  2013     1     1      557            600        -3
10  2013     1     1      558            600        -2
# … with 336,766 more rows, and 13 more variables:
```

`flights` is a tibble, a special type of data frame used by the `tidyverse`to avoid some common pitfalls. The most important difference between tibbles and data frames is the way tibbles print; they are designed for large datasets, so they only show the first few rows and only the columns that fit on one screen. There are a few options to see everything. If you're using `RStudio`, the most convenient is probably View(flights), which will open an interactive scrollable and filterable view.

```
View(flights)
```

The variables names are followed by abbreviations that tell you the type of each variable: `<int>` is short for integer, `<dbl>` is short for double (aka real numbers), `<chr>` for character (aka strings), and <`dttm`> for date-time. These are important because the operations you can perform on a column depend so much on its "type".

3 DPLYR BASICS

We're about to learn the primary `dplyr`verbs (functions) which will allow you to solve the vast majority of your data manipulation challenges. But before we discuss their individual differences, it's worth stating what they have in common:

1. The first argument is always a data frame.
2. The subsequent arguments typically describe which columns to operate on, using the variable names (without quotes).
3. The output is always a new data frame.

Because each verb does one thing well, solving complex problems will usually require combining multiple verbs, and we'll do so with the pipe, `|>`. We'll discuss the pipe more later, but in brief, the pipe takes the thing on its left and passes it along to the function on its right. The easiest way to pronounce the pipe is "then". That makes it possible to get a sense of the following code even though you haven't yet learned the details:

```
flights |>
  filter(dest == "IAH") |>
  group_by(year, month, day) |>
  summarize(
    arr_delay = mean(arr_delay, na.rm = TRUE)
  )
```

```
`summarise()` has grouped output by 'year', 'month'.
You can override using the `.groups` argument.
# A tibble: 365 × 4
# Groups:   year, month [12]
    year month   day arr_delay
   <int> <int> <int>     <dbl>
 1  2013     1     1     17.8
 2  2013     1     2      7
 3  2013     1     3     18.3
 4  2013     1     4     -3.2
 5  2013     1     5     20.2
 6  2013     1     6      9.28
 7  2013     1     7     -7.74
 8  2013     1     8      7.79
 9  2013     1     9     18.1
10  2013     1    10      6.68
# … with 355 more rows
>
```

`dplyr`verbs are organized into four groups based on what they operate on: rows, columns, groups, or tables. In the following sections you'll learn the most important verbs for rows, columns, and groups.

4 Rows

The most important verbs that operate on rows of a dataset are `filter()`, which changes which rows are present without changing their order, and `arrange()`, which changes the order of the rows without changing which are present. Both functions only affect the rows, and the columns are left unchanged. We'll also discuss `distinct()` which finds rows with unique values but unlike `arrange()` and `filter()` it can also optionally modify the columns.

4.1 FILTER

`filter()` allows you to keep rows based on the values of the columns. The first argument is the data frame. The second and subsequent arguments are the conditions that must be true to keep the row. For example, we could find all flights that departed more than 120 minutes (two hours) late:

```
flights |>
  filter(dep_delay > 120)
```

```
> flights |>
+     filter(dep_delay > 120)
# A tibble: 9,723 × 19
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      848           1835       853
 2  2013     1     1      957            733       144
 3  2013     1     1     1114            900       134
 4  2013     1     1     1540           1338       122
 5  2013     1     1     1815           1325       290
 6  2013     1     1     1842           1422       260
 7  2013     1     1     1856           1645       131
 8  2013     1     1     1934           1725       129
 9  2013     1     1     1938           1703       155
10  2013     1     1     1942           1705       157
# … with 9,713 more rows, and 13 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>
```

As well as `>` (greater than), you can use `>=` (greater than or equal to), `<` (less than), `<=` (less than or equal to), `==` (equal to), and `!=` (not equal to). You can also combine conditions with `&` or `,` to indicate "and" (check for both conditions) or with `|` to indicate "or" (check for either condition). To find flights that departed on January 1st.

```
flights |>
  filter(month == 1 & day == 1)
```

```
> flights |>
+     filter(month == 1 & day == 1)
# A tibble: 842 × 19
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      517            515         2
 2  2013     1     1      533            529         4
 3  2013     1     1      542            540         2
 4  2013     1     1      544            545        -1
 5  2013     1     1      554            600        -6
 6  2013     1     1      554            558        -4
 7  2013     1     1      555            600        -5
 8  2013     1     1      557            600        -3
 9  2013     1     1      557            600        -3
10  2013     1     1      558            600        -2
# … with 832 more rows, and 13 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>
```

To find flights that departed in January or February

```
flights |>
  filter(month == 1 | month == 2)
```

```
> flights |>
+     filter(month == 1 | month == 2)
# A tibble: 51,955 × 19
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      517            515         2
 2  2013     1     1      533            529         4
 3  2013     1     1      542            540         2
 4  2013     1     1      544            545        -1
 5  2013     1     1      554            600        -6
 6  2013     1     1      554            558        -4
 7  2013     1     1      555            600        -5
 8  2013     1     1      557            600        -3
 9  2013     1     1      557            600        -3
10  2013     1     1      558            600        -2
# … with 51,945 more rows, and 13 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>
```

There's a useful shortcut when you're combining `|` and `==`: `%in%`. It keeps rows where the variable equals one of the values on the right:

```
flights |>
  filter(month %in% c(1, 2))
```

```
> flights |>
+     filter(month %in% c(1, 2))
# A tibble: 51,955 × 19
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      517            515         2
 2  2013     1     1      533            529         4
 3  2013     1     1      542            540         2
 4  2013     1     1      544            545        -1
 5  2013     1     1      554            600        -6
 6  2013     1     1      554            558        -4
 7  2013     1     1      555            600        -5
 8  2013     1     1      557            600        -3
 9  2013     1     1      557            600        -3
10  2013     1     1      558            600        -2
# … with 51,945 more rows, and 13 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>
```

When you run `filter()` `dplyr`executes the filtering operation, creating a new data frame, and then prints it. It doesn't modify the existing flights dataset because `dplyr`functions never modify their inputs. To save the result, you need to use the assignment operator, `<-`:

```
jan1 <- flights |>
  filter(month == 1 & day == 1)
```

```
> jan1 <- flights |>
+     filter(month == 1 & day == 1)
> jan1
# A tibble: 842 × 19
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      517            515         2
 2  2013     1     1      533            529         4
 3  2013     1     1      542            540         2
 4  2013     1     1      544            545        -1
 5  2013     1     1      554            600        -6
 6  2013     1     1      554            558        -4
 7  2013     1     1      555            600        -5
 8  2013     1     1      557            600        -3
 9  2013     1     1      557            600        -3
10  2013     1     1      558            600        -2
# … with 832 more rows, and 13 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>
```

4.2 ARRANGE

`arrange()` changes the order of the rows based on the value of the columns. It takes a data frame and a set of column names (or more complicated expressions) to order by. If you provide more than one column name, each additional column will be used to break ties in the values of preceding columns. For example, the following code sorts by the departure time, which is spread over four columns. We get the earliest years first, then within a year the earliest months, etc.

```
flights |>
  arrange(year, month, day, dep_time)
```

```
#   minute <dbl>, time_hour <dttm>
> flights |>
+     arrange(year, month, day, dep_time)
# A tibble: 336,776 × 19
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      517            515         2
 2  2013     1     1      533            529         4
 3  2013     1     1      542            540         2
 4  2013     1     1      544            545        -1
 5  2013     1     1      554            600        -6
 6  2013     1     1      554            558        -4
 7  2013     1     1      555            600        -5
 8  2013     1     1      557            600        -3
 9  2013     1     1      557            600        -3
10  2013     1     1      558            600        -2
# … with 336,766 more rows, and 13 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>
```

You can use `desc()` on a column inside of `arrange()` to re-order the data frame based on that column in descending (big-to-small) order. For example, this code orders flights from most to least delayed:

```
flights |>
  arrange(desc(dep_delay))
```

```
#   minute <dbl>, time_hour <dttm>
> flights |>
+     arrange(desc(dep_delay))
# A tibble: 336,776 × 19
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     9      641            900      1301
 2  2013     6    15     1432           1935      1137
 3  2013     1    10     1121           1635      1126
 4  2013     9    20     1139           1845      1014
 5  2013     7    22      845           1600      1005
 6  2013     4    10     1100           1900       960
 7  2013     3    17     2321            810       911
 8  2013     6    27      959           1900       899
 9  2013     7    22     2257            759       898
10  2013    12     5      756           1700       896
# … with 336,766 more rows, and 13 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>
```

4.3 DISTINCT

`distinct()` finds all the unique rows in a dataset, so in a technical sense, it primarily operates on the rows. Most of the time, however, you'll want the distinct combination of some variables, so you can also optionally supply column names. To remove duplicate rows

```
flights |>
  distinct()
```

```
#   minute <dbl>, time_hour <dttm>
> flights |>
+     distinct()
# A tibble: 336,776 × 19
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      517            515         2
 2  2013     1     1      533            529         4
 3  2013     1     1      542            540         2
 4  2013     1     1      544            545        -1
 5  2013     1     1      554            600        -6
 6  2013     1     1      554            558        -4
 7  2013     1     1      555            600        -5
 8  2013     1     1      557            600        -3
 9  2013     1     1      557            600        -3
10  2013     1     1      558            600        -2
# … with 336,766 more rows, and 13 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>
```

Or, to find all unique origin and destination pairs.

```
flights |>
  distinct(origin, dest)
```

```
#   minute <dbl>, time_hour <dttm>
> flights |>
+     distinct(origin, dest)
# A tibble: 224 × 2
   origin dest
   <chr>  <chr>
 1 EWR    IAH
 2 LGA    IAH
 3 JFK    MIA
 4 JFK    BQN
 5 LGA    ATL
 6 EWR    ORD
 7 EWR    FLL
 8 LGA    IAD
 9 JFK    MCO
10 LGA    ORD
# … with 214 more rows
>
>
```

Alternatively, if you want to the keep other columns when filtering for unique rows, you can use the `.keep_all = TRUE` option.

```
flights |>
  distinct(origin, dest, .keep_all = TRUE)
```

```
> flights |>
+     distinct(origin, dest, .keep_all = TRUE)
# A tibble: 224 × 19
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      517            515         2
 2  2013     1     1      533            529         4
 3  2013     1     1      542            540         2
 4  2013     1     1      544            545        -1
 5  2013     1     1      554            600        -6
 6  2013     1     1      554            558        -4
 7  2013     1     1      555            600        -5
 8  2013     1     1      557            600        -3
 9  2013     1     1      557            600        -3
10  2013     1     1      558            600        -2
# … with 214 more rows, and 13 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>
>
```

It's not a coincidence that all of these distinct flights are on January 1: `distinct()` will find the first occurrence of a unique row in the dataset and discard the rest.

If you want to find the number of occurrences instead, you're better off swapping `distinct()` for `count()`, and with the `sort = TRUE` argument you can arrange them in descending order of number of occurrences.

```
flights |>
  count(origin, dest, sort = TRUE)
```

```
#   minute <dbl>, time_hour <dttm>
> flights |>
+     count(origin, dest, sort = TRUE)
# A tibble: 224 × 3
   origin dest      n
   <chr>  <chr> <int>
 1 JFK    LAX   11262
 2 LGA    ATL   10263
 3 LGA    ORD    8857
 4 JFK    SFO    8204
 5 LGA    CLT    6168
 6 EWR    ORD    6100
 7 JFK    BOS    5898
 8 LGA    MIA    5781
 9 JFK    MCO    5464
10 EWR    BOS    5327
# … with 214 more rows
>
```

5 Columns

There are four important verbs that affect the columns without changing the rows: `mutate()` creates new columns that are derived from the existing columns, `select()` changes which columns are present, `rename()` changes the names of the columns, and `relocate()` changes the positions of the columns.

5.1 mutate

The job of `mutate()` is to add new columns that are calculated from the existing columns. There is a large set of functions that you can use to manipulate different types of variables. For now, we'll stick with basic algebra, which allows us to compute the gain, how much time a delayed flight made up in the air, and the speed in miles per hour:

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    speed = distance / air_time * 60
  )
```

```
> flights |>
+     mutate(
+         gain = dep_delay - arr_delay,
+         speed = distance / air_time * 60
+     )
# A tibble: 336,776 × 21
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      517            515         2
 2  2013     1     1      533            529         4
 3  2013     1     1      542            540         2
 4  2013     1     1      544            545        -1
 5  2013     1     1      554            600        -6
 6  2013     1     1      554            558        -4
 7  2013     1     1      555            600        -5
 8  2013     1     1      557            600        -3
 9  2013     1     1      557            600        -3
10  2013     1     1      558            600        -2
# … with 336,766 more rows, and 15 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>, gain <dbl>,
#   speed <dbl>
>
```

By default, `mutate()` adds new columns on the right hand side of your dataset, which makes it difficult to see what's happening here. We can use the `.before` argument to instead add the variables to the left hand side

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    speed = distance / air_time * 60,
    .before = 1
  )
```

```
> flights |>
+     mutate(
+         gain = dep_delay - arr_delay,
+         speed = distance / air_time * 60,
+         .before = 1
+     )
# A tibble: 336,776 × 21
    gain speed  year month   day dep_time sched_dep_time
   <dbl> <dbl> <int> <int> <int>    <int>          <int>
 1    -9  370.  2013     1     1      517            515
 2   -16  374.  2013     1     1      533            529
 3   -31  408.  2013     1     1      542            540
 4    17  517.  2013     1     1      544            545
 5    19  394.  2013     1     1      554            600
 6   -16  288.  2013     1     1      554            558
 7   -24  404.  2013     1     1      555            600
 8    11  259.  2013     1     1      557            600
 9     5  405.  2013     1     1      557            600
10   -10  319.  2013     1     1      558            600
# … with 336,766 more rows, and 14 more variables:
#   dep_delay <dbl>, arr_time <int>,
#   sched_arr_time <int>, arr_delay <dbl>,
#   carrier <chr>, flight <int>, tailnum <chr>,
#   origin <chr>, dest <chr>, air_time <dbl>,
#   distance <dbl>, hour <dbl>, minute <dbl>,
#   time_hour <dttm>
>
```

The `.` is a sign that .before is an argument to the function, not the name of a third new variable we are creating. You can also use `.after` to add after a variable, and in both `.before` and `.after` you can use the variable name instead of a position. For example, we could add the new variables after day:

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    speed = distance / air_time * 60,
    .after = day
  )
```

```
> flights |>
+     mutate(
+         gain = dep_delay - arr_delay,
+         speed = distance / air_time * 60,
+         .after = day
+     )
# A tibble: 336,776 × 21
    year month   day  gain speed dep_time sched_dep_time
   <int> <int> <int> <dbl> <dbl>    <int>          <int>
 1  2013     1     1    -9  370.      517            515
 2  2013     1     1   -16  374.      533            529
 3  2013     1     1   -31  408.      542            540
 4  2013     1     1    17  517.      544            545
 5  2013     1     1    19  394.      554            600
 6  2013     1     1   -16  288.      554            558
 7  2013     1     1   -24  404.      555            600
 8  2013     1     1    11  259.      557            600
 9  2013     1     1     5  405.      557            600
10  2013     1     1   -10  319.      558            600
# … with 336,766 more rows, and 14 more variables:
#   dep_delay <dbl>, arr_time <int>,
#   sched_arr_time <int>, arr_delay <dbl>,
#   carrier <chr>, flight <int>, tailnum <chr>,
#   origin <chr>, dest <chr>, air_time <dbl>,
#   distance <dbl>, hour <dbl>, minute <dbl>,
#   time_hour <dttm>
>
```

Alternatively, you can control which variables are kept with the `.keep` argument. A particularly useful argument is "used" which specifies that we only keep the columns that were involved or created in the `mutate()` step. For example, the following output will contain only the variables `dep_delay`, `arr_delay`, `air_time`, `gain`, `hours`, and `gain_per_hour`.

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    hours = air_time / 60,
    gain_per_hour = gain / hours,
    .keep = "used"
  )
```

```
> flights |>
+     mutate(
+         gain = dep_delay - arr_delay,
+         hours = air_time / 60,
+         gain_per_hour = gain / hours,
+         .keep = "used"
+     )
# A tibble: 336,776 × 6
   dep_delay arr_delay air_time  gain hours
       <dbl>     <dbl>    <dbl> <dbl> <dbl>
 1         2        11      227    -9 3.78
 2         4        20      227   -16 3.78
 3         2        33      160   -31 2.67
 4        -1       -18      183    17 3.05
 5        -6       -25      116    19 1.93
 6        -4        12      150   -16 2.5
 7        -5        19      158   -24 2.63
 8        -3       -14       53    11 0.883
 9        -3        -8      140     5 2.33
10        -2         8      138   -10 2.3
# … with 336,766 more rows, and 1 more variable:
#   gain_per_hour <dbl>
>
```

Note that since we haven't assigned the result of the above computation back to `flights`, the new variables `gain`, `hours`, and `gain_per_hour` will only be printed but will not be stored in a data frame. And if we want them to be available in a data frame for future use, we should think carefully about whether we want the result to be assigned back to `It's not uncommon to get datasets with hundreds or even thousands of variables. In this situation, the first challenge is often just focusing on the variables you're interested in. select() allows you to rapidly zoom in on a useful subset using operations based on the names of the variables:flights`, overwriting the original data frame with many more variables, or to a new object. Often, the right answer is a new object that is named informatively to indicate its contents, e.g., `delay_gain`, but you might also have good reasons for overwriting `flights`.

5.2 SELECT

It's not uncommon to get datasets with hundreds or even thousands of variables. In this situation, the first challenge is often just focusing on the variables you're interested in. `select()` allows you to rapidly zoom in on a useful subset using operations based on the names of the variables:

- Select columns by name:

  ```
  flights |>
  select(year, month, day)
  ```

- Select all columns between year and day (inclusive):

```
flights |>
  select(year:day)
```

- Select all columns except those from year to day (inclusive):

```
flights |>
  select(!year:day)
```

  You can also use `-` instead of `!` we recommend `!` because it reads as "not", and combines well with `&` and `|`.

- Select all columns that are characters:

```
flights |>
  select(where(is.character))
```

There are a number of helper functions you can use within `select()`:

- `starts_with("abc")`: matches names that begin with "abc".
- `ends_with("xyz")`: matches names that end with "xyz".
- `contains("ijk")`: matches names that contain "ijk".
- `num_range("x", 1:3)`: matches x1, x2 and x3.

See `?select` for more details.

You can also rename variables as you `select()` them by using `=`. The new name appears on the left hand side of the `=`, and the old variable appears on the right hand side:

```
flights |>
  select(TailNumber = tailnum)
```

```
> flights |>
+     select(TailNumber = tailnum)
# A tibble: 336,776 × 1
   TailNumber
   <chr>
 1 N14228
 2 N24211
 3 N619AA
 4 N804JB
 5 N668DN
 6 N39463
 7 N516JB
 8 N829AS
 9 N593JB
10 N3ALAA
# … with 336,766 more rows
>
```

5.3 RENAME

If you want to keep all the existing variables and just want to rename a few, you can use `rename()` instead of `select()`:

```
flights |>
  rename(TailNumber = tailnum)
```

If you have a bunch of badly named columns (as can often happen in real data sets) and it would be painful to fix them all by hand, check out `janitor::clean_names()` which provides some useful automated cleaning.

5.4 RELOCATE

Use `relocate()` to move variables around. You might want to collect related variables together or move important variables to the front. By default `relocate()` moves variables to the front:

```
flights |>
  relocate(time_hour, air_time)
```

You can also specify where to put them using the `.before` and `.after` arguments, just like in `mutate()`:

```
flights |>
  relocate(year:dep_time, .after = time_hour)
flights |>
  relocate(starts_with("arr"), .before = dep_time)
```

6 The Pipe

We've shown you simple examples of the pipe above, but its real power arises when you start to combine multiple verbs. For example, imagine that you wanted to find the fast flights to Houston's IAH airport: you need to combine `filter()`, `mutate()`, `select()`, and `arrange()`:

```
flights |>
  filter(dest == "IAH") |>
  mutate(speed = distance / air_time * 60) |>
  select(year:day, dep\_time, carrier, flight, speed) |>
  arrange(desc(speed))
```

```
> flights |>
+     filter(dest == "IAH") |>
+     mutate(speed = distance / air_time * 60) |>
+     select(year:day, dep_time, carrier, flight, speed)
+     arrange(desc(speed))
# A tibble: 7,198 × 7
    year month   day dep_time carrier flight speed
   <int> <int> <int>    <int> <chr>    <int> <dbl>
 1  2013     7     9      707 UA         226  522.
 2  2013     8    27     1850 UA        1128  521.
 3  2013     8    28      902 UA        1711  519.
 4  2013     8    28     2122 UA        1022  519.
 5  2013     6    11     1628 UA        1178  515.
 6  2013     8    27     1017 UA         333  515.
 7  2013     8    27     1205 UA        1421  515.
 8  2013     8    27     1758 UA         302  515.
 9  2013     9    27      521 UA         252  515.
10  2013     8    28      625 UA         559  515.
# … with 7,188 more rows
>
```

Even though this pipeline has four steps, it's easy to skim because the verbs come at the start of each line: start with the `flights` data, then `filter`, then `mutate`, then `select`, then `arrange`.

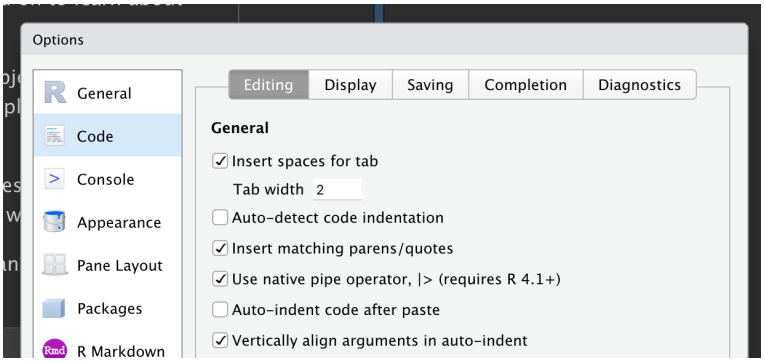
What would happen if we didn't have the pipe? We could nest each function call inside the previous call:

```
arrange(
  select(
    mutate(
      filter(
        flights,
        dest == "IAH"
      ),
      speed = distance / air_time * 60
    ),
    year:day, dep_time, carrier, flight, speed
  ),
  desc(speed)
)
```

Or we could use a bunch of intermediate objects:

```
flights1 <- filter(flights, dest == "IAH")
flights2 <- mutate(flights1, speed = distance / air_time * 60)
flights3 <- select(flights2, year:day, dep_time, carrier, flight, speed)
arrange(flights3, desc(speed))
```

To add the pipe to your code, we recommend using the build-in keyboard shortcut Ctrl/Cmd + Shift + M. You'll need to make one change to your `RStudio` options to use `|>` instead of `%>%`,more on %>% shortly.



7 Groups

So far you've learned about functions that work with rows and columns. `dplyr`gets even more powerful when you add in the ability to work with groups. In this section, we'll focus on the most important functions: `group_by()`, `summarize()`, and the `slice` family of functions.

7.1 group_by

Use `group_by()` to divide your dataset into groups meaningful for your analysis:

```
flights |>
  group_by(month)
```

```
> flights |>
+     group_by(month)
# A tibble: 336,776 × 19
# Groups:   month [12]
    year month   day dep_time sched_dep_time dep_delay
   <int> <int> <int>    <int>          <int>     <dbl>
 1  2013     1     1      517            515         2
 2  2013     1     1      533            529         4
 3  2013     1     1      542            540         2
 4  2013     1     1      544            545        -1
 5  2013     1     1      554            600        -6
 6  2013     1     1      554            558        -4
 7  2013     1     1      555            600        -5
 8  2013     1     1      557            600        -3
 9  2013     1     1      557            600        -3
10  2013     1     1      558            600        -2
# … with 336,766 more rows, and 13 more variables:
#   arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>,
#   minute <dbl>, time_hour <dttm>
>
```

`group_by()` doesn't change the data but, if you look closely at the output, you'll notice that the output indicates that it is "grouped by" month (Groups: month [12]). This means

subsequent operations will now work "by month". `group_by()` adds this grouped feature (referred to as class) to the data frame, which changes the behavior of the subsequent verbs applied to the data.

7.2 SUMMARIZE

The most important grouped operation is a summary, which, if being used to calculate a single summary statistic, reduces the data frame to have a single row for each group. In `dplyr`, this operation is performed by `summarize()`, as shown by the following example, which computes the average departure delay by month:

```
flights |>
  group_by(month) |>
  summarize(
    delay = mean(dep_delay, na.rm = TRUE)
  )
```

```
> flights |>
+     group_by(month) |>
+     summarize(
+         delay = mean(dep_delay, na.rm = TRUE)
+     )
# A tibble: 12 × 2
   month delay
   <int> <dbl>
 1     1 10.0
 2     2 10.8
 3     3 13.2
 4     4 13.9
 5     5 13.0
 6     6 20.8
 7     7 21.7
 8     8 12.6
 9     9  6.72
10    10  6.24
11    11  5.44
12    12 16.6
>
```

You can create any number of summaries in a single call to `summarize()`. You'll learn various useful summaries in the upcoming chapters, but one very useful summary is `n()`, which returns the number of rows in each group:

```
flights |>
  group_by(month) |>
  summarize(
    delay = mean(dep_delay, na.rm = TRUE),
    n = n()
  )
```

```
> flights |>
+     group_by(month) |>
+     summarize(
+         delay = mean(dep_delay, na.rm = TRUE),
+         n = n()
+     )
# A tibble: 12 × 3
   month delay     n
   <int> <dbl> <int>
 1     1 10.0  27004
 2     2 10.8  24951
 3     3 13.2  28834
 4     4 13.9  28330
 5     5 13.0  28796
 6     6 20.8  28243
 7     7 21.7  29425
 8     8 12.6  29327
 9     9  6.72 27574
10    10  6.24 28889
11    11  5.44 27268
12    12 16.6  28135
>
```

A huge range of options are vailable to work with groups. For a full list please read Section 4 Data transformation in R for Data Science.